

LABORATORY EXPERIMENTAL PROTOCOL — Completed

Applied Massive Parallelism: CUDA Python, Thread Geometries, and Hardware Latency Pitfalls
Parallel Computing Lab | Zeba Academy / GPU Architecture & CUDA Python

Student Name: **Kystaubay Ayanat**

Student ID: **230103075**

Session / Date: **08.10.2026, 75-minute session — Google Colab, T4 GPU runtime**

Assigned GPU ID: **GPU 0, Bus-Id 00000000:00:04.0 (Tesla T4)**

Notebook: **CUDA_Lab01_230103075.ipynb (all cells executed sequentially, outputs retained)**

Section 1 — Hardware Telemetry Extraction

Mandatory submission field	Recorded value
NVIDIA Driver Version (from nvidia-smi)	580.82.07
CUDA Version Supported by Driver	13.0
Allocated GPU Name	Tesla T4
Total Available VRAM	15360 MiB reported by nvidia-smi (14913 MiB visible to the CUDA context, 14808 MiB free)
Compute Capability Major.Minor	7.5 (Turing)
Max Threads Per Block (hardware limit)	1024

Additional telemetry from the Numba CUDA driver interface: Multiprocessor Count (SMs) = **40**; Max Block Dimensions = **1024, 1024, 64**; Warp Size = **32**; temperature at launch 57-71 C, power draw 30-33 W of a 70 W cap.

Section 2 — The Crossover Experiment (PCIe latency vs. compute)

Array Size (N)	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner
100	0.084	1.316	0.0909	CPU
1,000	0.252	0.567	0.0709	CPU
10,000	2.649	0.570	0.0774	GPU
100,000	28.545	0.754	0.0867	GPU
1,000,000	358.234	7.707	0.1795	GPU
5,000,000	skipped (CPU loop too slow)	17.703	0.3120	GPU
10,000,000	skipped (CPU loop too slow)	39.548	0.5285	GPU

Q1. Empirical Crossover Point N*

N* = 10,000. At N = 1,000 the CPU still wins (0.252 ms vs 0.567 ms), while at N = 10,000 the GPU roundtrip (0.570 ms) becomes 4.6x faster than the pure-Python loop (2.649 ms). Past that point the gap widens rapidly — at N = 1,000,000 the GPU is 46x faster (7.707 ms vs 358.234 ms) — because the CPU cost grows linearly with N while the GPU roundtrip is dominated by a nearly constant launch and transfer overhead.

Q2. Why GPU Total is dramatically slower at N = 100

At N = 100 the GPU roundtrip took 1.316 ms while the kernel itself needed only 0.0909 ms, so **93.1% of the elapsed time was not computation at all**. That time went into the fixed costs that surround every GPU call: allocating the device buffers, the host-to-device copy of 400 bytes across the PCIe bus, the kernel launch itself (the driver must build the launch packet, push it to the command queue and have the GPU schedule a grid), the implicit synchronisation, and finally the device-to-host copy back. A PCIe transaction costs several microseconds of pure latency regardless of payload size, so moving 400 bytes costs almost exactly as much as moving 400 KB — the bus is latency-bound, not bandwidth-bound, at this scale. Meanwhile the CPU needs only 0.084 ms to touch 100 floats that are already in its L1 cache, with no transfer at all. The GPU wins only once the array is large enough for the parallel work to outweigh that fixed overhead, which on this machine happens at N = 10,000.

Section 3 — Thread Geometry & Boundary Memory Corruption

Task 3.1 — Observed behaviour of the defective kernel

Console output recorded on my instance:

Observed behaviour / console output from the defective kernel
Launching defective kernel with 1024 threads on 1000-element array... Execution finished (or did it crash?). Checking host copy... Elements processed: 1000. Last element: 999.0 No Python exception was raised — execution completed normally and <code>cuda.synchronize()</code> returned without error.

No `IndexError`, no `CUDA_ERROR_ILLEGAL_ADDRESS`, no crash: the 24 out-of-range threads (global IDs 1000-1023) wrote 999.0 into raw VRAM immediately after the 4000-byte buffer. CUDA device memory has no bounds metadata — a kernel writes to an address, and the MMU only objects if the address leaves the allocated page entirely. Numba had padded the allocation, so the stray writes landed inside memory the driver had already mapped and corrupted it silently. This is the dangerous case: the program appears to work, the visible 1000 elements are correct, and the corruption would only surface later as wrong values in some unrelated buffer that happened to live at that address. The guard `"if idx < n"` is what prevents this.

Task 3.2 — Spatial thread coordinate inspector (N = 37, block size 8)

Global ID	Block ID	Thread ID	Status
32	4	0	ACTIVE
33	4	1	ACTIVE
34	4	2	ACTIVE
35	4	3	ACTIVE
36	4	4	ACTIVE
37	4	5	IDLE (GUARDED)
38	4	6	IDLE (GUARDED)
39	4	7	IDLE (GUARDED)

Total Blocks Allocated	Total Hardware Threads Launched	Active Working Threads	Idle Guarded Threads
5	40	37	3

Ceiling division gives $\text{ceil}(37 / 8) = 5$ blocks = 40 threads for 37 elements. The boundary falls inside block 4: threads 0-4 of that block map to global IDs 32-36 and do real work, while threads 5-7 map to 37-39 and are switched off by the bounds check. Global ID = $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$, which the table confirms ($4 * 8 + 5 = 37$).

Section 4 — The Race Condition Crime Scene (Parallel Reduction)

Trial #	Target Sum	Naive Kernel Output	Deviation / Lost Updates
Trial 1	50,000.0	2.0	49,998.0
Trial 2	50,000.0	2.0	49,998.0
Trial 3	50,000.0	2.0	49,998.0
Trial 4	50,000.0	2.0	49,998.0
Trial 5	50,000.0	2.0	49,998.0

- Atomic Sum Output: **50000.0** — exactly 50,000.0? YES
- Naive Kernel Time: **0.0354 ms** vs. Atomic Kernel Time: **0.1876 ms** (atomic is 5.31x slower)

The naive kernel lost 99.996% of the data: 50,000 threads each read `total[0]`, added 1.0 and wrote the result back, but because all of them read the same value at nearly the same clock cycle, essentially every write overwrote another thread's. The surviving value of 2.0 is not noise — it is the handful of updates that happened to be the last writers.

Why atomic operations incur a massive execution penalty

`cuda.atomic.add` compiles to a hardware read-modify-write that the memory subsystem must execute indivisibly. The L2 cache controller takes exclusive ownership of the target 32-byte sector, performs the addition and releases it, so every thread addressing the same word is serialised by the hardware. With 50,000 threads hammering a single address, the entire reduction degenerates into a queue at one L2 bank: the 1,562 warps can no longer run in parallel for that instruction, and warp schedulers stall waiting for their turn. There is also no coalescing to exploit — normally 32 threads of a warp touch 32 consecutive addresses in one transaction, whereas here they all touch the same one and the conflict must be resolved sequentially. That is why my atomic kernel took 0.1876 ms against 0.0354 ms for the racing version: the naive kernel is fast precisely because it is wrong — it never waits for anyone. The production answer is neither: a tree reduction in shared memory per block, followed by one atomic per block (here 196 atomics instead of 50,000),

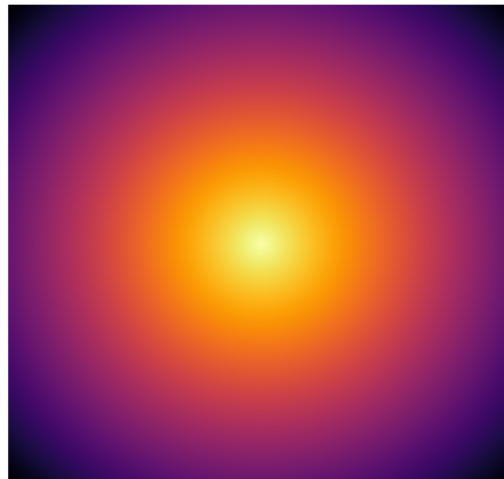
which keeps correctness while restoring parallelism.

Section 5 — 2D Grid Image / Matrix Processing

Blocks in X	Blocks in Y	Total Blocks	Threads per block	Center (512, 512)	Corner (0, 0)
64	64	4096	16 x 16 = 256	1.000000	0.000000

Grid geometry: $\text{ceil}(1024 / 16) = 64$ blocks per axis, $64 \times 64 = 4,096$ blocks of 256 threads = 1,048,576 threads, exactly one per pixel of the 1024 x 1024 canvas. The centre pixel sits at distance 0 from (512, 512), so its intensity is $1 - 0 / \text{max_dist} = 1.0$; the corner (0, 0) lies at distance $\sqrt{512^2 + 512^2}$, which equals max_dist exactly, so the intensity falls to 0.0. Generated matrix shape: (1024, 1024).

Radial vignette 1024x1024 (2D grid)



Section 6 — Final Verification Checksum

Empirical crossover N* used as seed	Student ID	16-character verification token
10000	230103075	568F4EA22E3218E5

Final submission checklist:

- Notebook saved as CUDA_Lab01_230103075.ipynb — done
- All code cells executed sequentially with outputs visible (no cleared outputs) — done
- Exported as .ipynb (File → Download → Download .ipynb) — done
- This task sheet filled out completely — done